

어셈블리프로그램 설계 및 실습

Term Project 보고서



담당교수	교수님
실습분반	목요일 (1분반)
소속	컴퓨터정보공학부
학번·성명	2022202109 이호정
제출일	2024년 12월 10일

1. Problem Statement

이 과제의 목적은 MNIST 숫자 데이터의 이미지를 ARM 어셈블리 언어로 확대하는 프로그램을 구현하는 것입니다. 20x20 크기의 입력 이미지를 Bilinear Interpolation 알고리즘을 사용해 80x80 크기로 확장하는 프로그램을 작성해야 하며, 입력 데이터는 IEEE 754 Single Precision 형식으로 저장되어 있습니다. 결과물은 품질(PSNR) 및 연산 성능(코드 크기와 상태의 곱)으로 평가됩니다.

이 프로그램은 선형보간법을 활용하여 20x20 이미지를 80x80으로 확대하는 프로그램이다.

2. Algorithm

구현방법을 여러가지 생각해보았다.

- 방법1

Bilinear interpolation 공식을 활용하는 방법이다.

먼저 80x80 행렬에 올바른 값을 계산하여 입력하고 저장하는 방식이다. 먼저 80x80을 순회하며 i열 과 j 열 좌표가 있으면, 그 좌표를 4로 나눈다. 그러면 x1과 y1 좌표값을 구할 수있다. 4로 나누었을 때 정수부만 저장하기 때문에 해당 값들을 x1, y1으로 저장한다. 그리고 해당 좌표에 +1 즉, 한칸 옮긴 좌표값이 각각 x2, y2 이기 때문에 이를 이용하여 점 P의 값을 계산할 수식에 필요한 좌표 4개를 도출해 낼 수 있다.

Q11 : 좌하단 좌표로 (x1, y1) 으로 나타낼 수 있다.

Q21 : 우하단 좌표로 (x2, y1) 으로 나타낼 수 있다.

Q12 : 좌상단 좌표로 (x2, y1) 으로 나타낼 수 있다.

Q22 : 우상단 좌표로 (x2, y2) 으로 나타낼 수 있다.

이렇게 각각의 좌표값을 구했다면 수식으로 P 의 값을 알아낼 수있다. 수식은 강의자료에 있는 것을 x 변화량을 dx 로 표현하고 , 그에 맞게 1-dx, dy, 1-dy 로 수정해 , 구현에 용이하도록 하였다.

$$P \rightarrow f(x,y) = Q11 * (1-dx) * (1-dy) + Q21 * dx * (1-dy) + Q12 * (1-dx) * dy + Q22 * dx * dy$$

이렇게 나타낼 수 있다. 그리고 해당 dx 는 어떻게 구하느냐?

모듈러 연산으로 구할 수 있다. 4로 나눈 나머지에 따라 맞는 가중치, 비율값을 반환하도록 코드를 구현하였다. 나올 수 있는 나머지는 0,1,2,3 이 있을 것이다. (0은 제외 가능)

따라서 해당 dx를 구하면 맞는 IEEE 값을 저장해뒀다가 반환하는 함수를 만들었다.

나머지 1 -> 가중치 0.25

나머지 2 -> 가중치 0.5

나머지 3 -> 가중치 0.75 이런식으로 구현하였다.

그리고 각각의 값들을 IEEE multiply 해주는 로직을 구현하면 가능하다. 다음은 해당 곱셈 로직에 대해 간략하게 설명해보겠다.

부동소수점 곱셈은 덧셈과 비슷하게 먼저 비트 분류하고 비트 추출하는게 포인트이다.

먼저, 두 값의 부호 비트를 추출하여 결과 값의 부호를 결정한다. 부호는 XOR 연산을 통해 계산하며, 두 값의 부호가 같으면 양수, 다르면 음수가 되도록 설정한다.

그다음으로, 두 값의 지수를 추출한다. 이때 지수는 부동소수점 값에서 127의 Bias 값을 빼야 실제 지수를 얻을 수 있다. 두 지수를 더한 뒤, 127을 다시 빼서 곱셈 결과의 지수를 계산한다. 가수를 처리하는 단계에서는 각 값의 가수를 추출하고, IEEE 754에서 암묵적으로 1로 가정되는 최상위 비트를 복원한다. 이렇게 하면 가수를 정규화된 형태로 만들어 줄 수 있다. 그 후 두 가수를 곱하는데, 32비트의 두 가수를 곱하면 64비트의 결과가 나오게 된다. 이 연산은 ARM의 UMULL 명령어를 사용하여 수행하며, 곱셈의 상위 32비트와 하위 32비트를 얻을 수 있도록 구성한다.

가수를 곱한 후에는 결과 가수가 정규화된 형태인지 확인한다. 정규화되지 않은 경우, 가수가 너무 크면 오른쪽으로 시프트하고, 그에 따라 지수를 하나 증가시킨다. 반대로 가수가 너무 작으면 왼쪽으로 시프트하고, 지수를 하나 감소시키며 정규화한다. 이를 통해 가수를 1.xx의 형태로 유지하도록 조정한다.

마지막으로, 계산된 부호, 지수, 가수를 조합하여 최종 결과를 생성한다. 가수는 하위 23비트로 제한하고, 지수는 올바른 위치에 배치하며, 부호는 최상위 비트에 추가하여 결과 값을 구성한다.

- 방법 2

먼저 데이터 확장부터한다. 기존 20x20 데이터를 80x80 짜리 공간에 먼저 알맞게 삽입한다. 그때에는 주소값을 늘려주면서 픽셀을 이동하고 값을 저장하는 방식을 사용한다.

20x20 에서 한칸 이동한다는 것은 80x80에서는 4칸 이동하는 것을 의미한다. 따라서 이와 같은 방식으로 이동하고 값을 저장한다. [r5 , #16], 이렇게 하면 기존 데이터에서 한칸 이동한 값을 확장한곳에 저장할 수있다. 그리고 다음 행으로 가는 것은 +1280 하는 방식으로 구현할 수있다.

- 방식3

선형보간이기 때문에 증가하는 값이 같을 것이다. 따라서 Q21-Q11 한 값을 4개로 나누고 그 값을 한 칸 에 한번씩 더해주는 것이다 . 이렇게 하면 multiply 없이 add 와 sub 만으로도 구현할 수 있다. 4분의 1은 0.25이니 연산 가능하다.

3. Result

● Q11

Register list (Current):

Register	Value
R0	0x10006410
R1	0x00000000
R2	0x00000000
R3	0x00000000
R4	0x00000440
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x3F800000

Assembly code (projects):

```

37: MUL R3, R8, R0
38: ADD R3, R3, R9
39: LDR R10, [R4, R3, LSL #2] ; Load pixel value for Q11
40: LDR R0, =store_q11_x      ; Address of Q11 x-coordinate
41: STR R10, [R0]             ; Store Q11 x-coordinate
42:

```

→ q11 좌표 픽셀의 값이 올바르게 저장되는 것을 확인할 수 있다.

Register list (Current):

Register	Value
R0	0x10006410
R1	0x00000000
R2	0x00000000
R3	0x00000000
R4	0x00000440
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x10006410

Assembly code (projects):

```

41: STR R10, [R0] ; Store Q11 x-coordinate
42:
43: ; Q12 (Bottom Right)
44: STR R10, [R0] ; Store Q12 x-coordinate

```

● Q12

Register list (Current):

Register	Value
R0	0x10006414
R1	0x00000000
R2	0x00000000
R3	0x00000001
R4	0x00000440
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000001
R10	0x10006414

Assembly code (projects):

```

51:
52: ; Q21 (Top Left)
53: STR R10, [R0] ; Store Q21 x-coordinate

```

→ q12 좌표 픽셀의 값이 올바르게 저장되는 것을 확인할 수 있다.

● Q21

Register list (Current):

Register	Value
R0	0x10006418
R1	0x00000000
R2	0x00000000
R3	0x00000014
R4	0x00000440
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000001
R9	0x00000000
R10	0x10006418

Assembly code (projects):

```

61:
62: ; Q22 (Top Right)
63: STR R10, [R0] ; Store Q22 x-coordinate

```

→ q21 좌표 픽셀의 값이 올바르게 저장되는 것을 확인할 수 있다.

● Q22

Register	Value
R0	0x1000641C
R1	0x00000000
R2	0x00000000
R3	0x00000015
R4	0x00000440
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000001
R9	0x00000001

```

69: STR R10, [R0] ; Store Q22 x-coordinate
70:
71: ;dx
0x00000094 E580A000 STR R10, [R0]
    
```

```

66: ADD R3, R3, R9
67: LDR R10, [R4, R3, LSL #2] ; Load pixel value for Q22
68: LDR R0, =store_q22_x ; Address of Q22 x-coordinate
69: STR R10, [R0] ; Store Q22 x-coordinate
    
```

→ q22 좌표 픽셀의 값이 올바르게 저장되는 것을 확인할 수 있다.

● 1-Dy

Register	Value
R0	0x3F800000
R1	0x00000434
R2	0x00000000
R3	0x00000015
R4	0x00000440
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000001
R9	0x00000001
R10	0x00000000
R11	0x00000000

```

93: STR R0, [R1] ; Store 1-dy
94:
    
```

```

89: ;1-dy
90: MOV R0, R0 ; Reuse R0
91: BL get_one_minus_dy ; Call get_one_minus_dy
92: LDR R1, =store_ldy ; Load address of store_ldy
93: STR R0, [R1] ; Store 1-dy
94:
    
```

→ 비율 (1-dy) 가 올바르게 계산되어서 r0 에 IEEE 형태로 저장된 것을 확인할 수 있다. 첫번째 픽셀이므로 dy 가 0이니 1-dy 가 1이 되고, 1의 IEEE 형태인 0x3F800000 이 저장된 것을 확인할 수 있다.

● Before Multiply

Register	Value
R0	0x3F800000
R1	0x1000640C
R2	0x00000000
R3	0x00000015
R4	0x10006404
R5	0x10000000
R6	0x00000000
R7	0x00000000
R8	0x00000001
R9	0x00000001
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13	0xFFFFFEC
R14	0x000000DC

```

222: BL multiply_float
0x000001D4 FB000021 BL 0x00000260
    
```

```

215: ; Q11(1-dx)(1-dy) + Q12(dx)(1-dy) + Q21(dx)(1-dy)
216:
217: ;Q11(1-dx)(1-dy)
218: LDR R4, =store_q11_x ; Address of store_q11_x
219: LDR R0, [R4]
220: LDR R4, =store_ldx
221: LDR R1, [R4]
222: BL multiply_float
    
```

→ 곱셈을 하기전 두 인자에 대한 정보가 올바르게 레지스터에 불러와진 것을 확인할 수 있다.

● Multiply

<pre> current R0 0x3F800000 R1 0x00000000 R2 0x00000000 R3 0x00000015 R4 0x10006404 R5 0x10000000 R6 0x00000000 R7 0x00000000 R8 0x00000001 R9 0x00000001 R10 0x00000000 R11 0x00000000 </pre>	<pre> 0x00000224 E5A00000 MOV R0,#0x00000000 328: POP {R4-R11, PC} 329: project.s 324 POP {R3-R11, PC} 325 326 zero_case 327 MOV R0, #0x00000000 ; Result is 0 328 POP {R4-R11, PC} 329 </pre>
--	--

→ Dy 가 0이므로 곱셈에서 특수 케이스 중 0을 반환되는 것을 확인할 수 있다.

<pre> current R0 0x00000000 R1 0x3F800000 R2 0x00000000 R3 0x00000000 R4 0x1000640C R5 0x10000000 R6 0x00000000 R7 0x00000000 R8 0x00000001 R9 0x00000001 R10 0x00000000 R11 0x00000000 </pre>	<pre> 228: ; Q12(dx) (1-dy) 0x000001F4 EB00001D BL 0x00000260 project.s 223 MOV R3, R0 ; r0 = Q11(1-dx) 224 LDR R4, =store_ldy 225 LDR R1, [R4] 226 BL multiply_float ; r0 = Q11(1-dx) (1-dy) 227 </pre>
--	--

→ 저장결과 0

● addition

<pre> current R0 0x00000000 R1 0x00000000 R2 0x00000000 R3 0x00000000 R4 0x00000000 R5 0x00000000 R6 0x00000000 R7 0x00000000 R8 0x00000001 R9 0x00000001 R10 0x00000000 R11 0x00000000 R12 0x00000000 R13 (...) 0xFFFFFC8 R14 (...) 0x00000210 R15 (...) 0x00000314 </pre>	<pre> 0x00000314 E1A06081 MOV R6,R1,LSL #1 349: MOV R6, R6, LSR #24 350: project.s 333 334 second_is_one 335 ; R0 already contains the correct value 336 POP {R4-R11, PC} 337 338 add_float 339 MOV R1,R11 ;prev result 340 341 PUSH {R3-R10, LR} ; Save registers </pre>
---	---

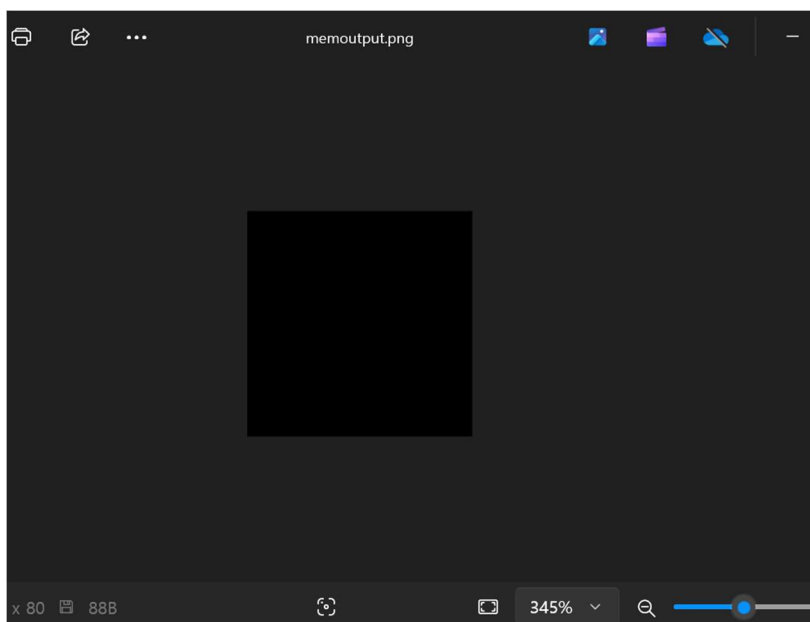
→ 덧셈 수행 후 0 이 올바르게 저장된 것을 확인할 수 있다.

- 0에 대한 출력



➔ 이미지가 올바르게 확대 출력되는 것을 확인할 수 있다.

- 나머지 출력은 올바르게 나오지 않아서, 오랫동안 코드 수정하고 연구했지만 실패했습니다.



4. Discussion and Conclusion

과제를 수행하며 느낀 점, 개선 방향, 과제 수행 중 발생한 문제점, 또는 과제에서 잘못 주어진 오류 등에 대한 분석과 결론을 작성합니다.

먼저 과제를 수행하는데에 정말 많은 시간을 투자하고, 많은 어려움이 있었다. 20x20 행렬 이미지 데이터를 80x80 로 확장한다는 구현 방법을 생각해 내는데에 먼저 오랜 시간이 걸렸다. 그리고 여러가지 구현 방법을 생각한 이유가 처음에 생각한 방식대로 구현이 되지 않아서 이다. 이렇게 전체 구현 방법을 생각해 내는데 정말 많은시간이 들었고 어려움이 있었다.

두번째로는 레지스터 관리가 어려웠다. C 언어로 코딩하던 버릇대로 코딩하다보니 레지스터가 너무 부족했다. 그래서 레지스터 사용에 대해 깊이 고민해 보고, 재사용해도 되는 레지스터와 재사용하면 안되는 레지스터를 구분해서 재사용 가능한 레지스터를 현명하게 재사용하는 방식으로 수정하였다. 이때도 레지스터 충돌이 일어나는지 고려해야하기 때문에 어려웠다. 그리고 그렇게 해도 안되는 부분은 메모리에 자리를 만들어두고 레지스터 하나로만 접근해 계속 값을 가져다 쓰고, 다음 루프때 또 덮어씌우고 레지스터로 값을 가져오고 지우고 하는 방식을 사용했다. 그리고 과제를 big ladian 으로 저장하는 방식에 있어서 오류가 있었는데 이 부분은 hex-decode.py 파일을 수정하니 이미지가 올바르게 생성되었다.

(Optional) Acknowledgement

레지스터 관리하는 방식에 대해 조언을 들었다. 레지스터 몇 개는 건들면 안되는 레지스터로 두고, 몇 개는 재사용 가능한 레지스터로 두어 관리하면 보다 쉽게 관리할 수 있다는 조언이 도움이 되었다.

Reference

해당 과목 pdf 9장

명예서약서 (Honor code)

“꿈을 가지십시오. 그리고 정열적이고 명예롭게 이루십시오.”

본인은 [어셈블리프로그램설계및실습] 수강에 있어 다음의 명예서약을 준수할 것을 서약합니다.

1. 기본 원칙

- 본인은 공학도로서 개인의 품위와 전문성을 지키며 행동하겠습니다.
- 본인은 안전, 건강, 공정성을 수호하는 책임감 있는 공학도가 되겠습니다.
- 본인은 우리대학의 명예를 지키고 신뢰받는 구성원이 되겠습니다.

2. 학업 윤리

- 모든 과제와 시험에서 정직하고 성실한 태도로 임하겠습니다.
- 타인의 지적 재산권을 존중하고, 표절을 하지 않겠습니다.
- 모든 과제는 처음부터 직접 작성하며, 타인의 결과물을 무단으로 사용하지 않겠습니다.
- 과제 수행 중 받은 도움이나 협력 사항을 보고서에 명시적으로 기재하겠습니다.

3. 서약 이행

본인은 위 명예서약의 모든 내용을 이해하였으며, 이를 성실히 이행할 것을 서약합니다.

학번 | 2022202109 | 이름 | 이호정 | 서명 | 이호정